

Q1. Cosa si intende per "spazio metrico" e in che modo la "metrica euclidea su $\mathbb{R}$ " vi si relaziona?

Uno **spazio metrico** è una coppia (X, d) con X insieme non vuoto e $d : X \times X \rightarrow \mathbb{R}$ (distanza) che per ogni x, y, z soddisfa 4 assiomi:

1. **Non-negatività:** $d(x, y) \geq 0$
2. **Identità:** $d(x, y) = 0 \Leftrightarrow x = y$
3. **Simmetria:** $d(x, y) = d(y, x)$
4. **Disuguaglianza triangolare:** $d(x, z) \leq d(x, y) + d(y, z)$

La **metrica euclidea su $\mathbb{R}$** è $d(x, y) = |x - y|$: soddisfa i 4 assiomi ed è il caso prototipo. Si generalizza a $\mathbb{R}^N$ con $d(x, y) = \|x - y\|_2 = \sqrt{\sum_i (x_i - y_i)^2}$. La metrica serve perché tutti i concetti di limite, continuità e ottimizzazione (es. discesa del gradiente in $\mathbb{R}^N$) richiedono una nozione di "vicinanza".

Q2. Come si definisce la "continuità" per una funzione @@INL35@@, e qual è il significato di "continuità separata"?

f è **continua** in x_0 se:

$$\forall \varepsilon > 0, \exists \delta > 0 : \|x - x_0\| < \delta \Rightarrow |f(x) - f(x_0)| < \varepsilon$$

Cioè: a piccole variazioni dell'input corrispondono piccole variazioni dell'output (definizione di Cauchy generalizzata da intervalli a "palle" in $\mathbb{R}^N$). La **continuità separata** richiede che f sia continua rispetto a una variabile alla volta, tenendo fisse le altre. **Relazione:** la continuità (congiunta) implica quella separata, **ma non viceversa**. Controesempio: $f(x, y) = xy/(x^2 + y^2)$ (e $f(0, 0) = 0$) è separatamente continua nell'origine (vale 0 sugli assi) ma non continua (vale 1/2 sulla retta $y = x$). È la prima prova che il calcolo in più variabili non è la semplice somma di calcoli a una variabile.

Q3. Qual è la relazione tra "derivata direzionale", "derivata parziale" e "gradiente" nel calcolo in più variabili?

- **Derivata direzionale** lungo il versore $\mathbf{v}$: $D_{\mathbf{v}}f(x_0) = \lim_{h \rightarrow 0} [f(x_0 + h\mathbf{v}) - f(x_0)]/h$. Misura la variazione di f muovendosi nella direzione $\mathbf{v}$.
- **Derivata parziale**: derivata direzionale lungo un versore della base canonica e_i (deriva rispetto a x_i , fissate le altre).
- **Gradiente**: vettore delle derivate parziali $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_n)$.

Relazione (se f differenziabile): $D_{\mathbf{v}}f = \nabla f \cdot \mathbf{v}$. Conseguenze:

- il gradiente contiene tutte le derivate direzionali;
- ∇f punta nella direzione di **massima crescita** (e $-\nabla f$ di massima decrescita), per Cauchy-Schwarz;
- $\nabla f = 0$ indica un punto critico.

Questa proprietà è il cuore della discesa del gradiente.

Q4. Descrivi il "metodo iterativo di discesa del gradiente" e la sua finalità.

Finalità: trovare un minimo di una funzione differenziabile f (es. la funzione di costo di una rete) quando non c'è soluzione in forma chiusa. **Idea:** poiché $-\nabla f$ è la direzione di massima decrescita, ci si muove a ogni passo in quella direzione. **Algoritmo:** scelto x_0 e un **learning rate** $\eta > 0$:

$$x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$$

fino a un criterio di arresto ($\|\nabla f\|$ piccola, max iterazioni). **Learning rate:** troppo grande $\rightarrow$ oscilla/diverge; troppo piccolo $\rightarrow$ lentissimo. **Garanzia:** converge al minimo globale solo se f è convessa; altrimenti a un punto critico. Varianti: Batch GD, SGD (un esempio per volta), Mini-batch.

Q5. Descrivi il metodo della discesa del gradiente. In quali casi potrebbe non convergere al minimo globale?

Metodo: $x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$ (vedi Q4). Garantisce solo la convergenza a un **punto critico**. Non raggiunge il minimo globale quando:

1. **Funzione non convessa con minimi locali** (tipico delle reti): si ferma nel primo minimo locale.
2. **Punti di sella** ($\nabla f = 0$ ma non minimo), frequenti in alta dimensione.
3. **Plateau** con gradiente quasi nullo: avanzamento bloccato (vanishing gradient).
4. **Learning rate sbagliato**: troppo grande $\rightarrow$ diverge; troppo piccolo $\rightarrow$ non arriva nel tempo utile.
5. **Inizializzazione sfortunata**: finisce nel bacino di un minimo locale scadente.

In sintesi: minimo globale garantito **solo se f è convessa**.

Q6. Come funziona un Single Layer Perceptron (SLP)? Quali sono i suoi limiti?

L'**SLP** ha ingressi $\mathbf{x}$, pesi $\mathbf{w}$, bias b e attivazione φ (storicamente il gradino). Calcola:

$$z = w \cdot x + b, y = \varphi(z)$$

Definisce un **iperpiano** $w \cdot x + b = 0$ e classifica i punti in base al lato in cui cadono: è un **classificatore lineare binario**. Si addestra con la regola del perceptron, che converge se i dati sono linearmente separabili. **Limiti**:

1. Risolve **solo problemi linearmente separabili**: NON lo XOR (Minsky-Papert).
2. Una sola superficie di decisione lineare: niente funzioni non lineari.
3. Il gradino non è differenziabile $\rightarrow$ niente backpropagation.

Si superano con il **MLP** (strati nascosti + attivazioni non lineari).

Q7. Confronta le funzioni di attivazione ReLU, sigmoide e tanh in termini di proprietà matematiche e applicazioni.

	Formula	Codominio	Note
Sigmoide	$\sigma(z) = 1/(1 + e^{-z})$	$(0,1)$	satura $\rightarrow$ vanishing gradient; non centrata in 0
tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	$(-1,1)$	centrata in 0; satura comunque
ReLU	$\max(0, z)$	$[0, \infty)$	no saturazione per $z > 0$; veloce; "dying ReLU" per $z < 0$

- **Sigmoide:** derivata $\sigma' = \sigma(1 - \sigma) \leq 1/4$; interpretabile come probabilità. Uso: output binario, gate LSTM.
- **tanh:** output centrato $\rightarrow$ gradienti più bilanciati della sigmoide. Uso: hidden layer RNN.
- **ReLU:** derivata 1 per $z > 0 \rightarrow$ niente vanishing gradient, induce sparsità. È lo **standard** negli hidden layer delle reti deep.

Q8. Spiega il principio dell'algoritmo di backpropagation in un MLP con un layer nascosto.

MLP con un layer nascosto:

$$h = \varphi(W^{(1)}x + b^{(1)}), \hat{y} = \psi(W^{(2)}h + b^{(2)})$$

La **backpropagation** calcola i gradienti della loss rispetto a tutti i parametri applicando la **regola della catena** all'indietro. **Forward**: calcola $z^{(1)}, h, z^{(2)}, \hat{y}$ e la loss L . **Backward** (propaga l'errore δ):

$$\begin{aligned}\delta^{(2)} &= \partial L / \partial \hat{y} \cdot \psi'(z^{(2)}) \partial L / \partial W^{(2)} = \delta^{(2)} h^T \\ \delta^{(1)} &= (W^{(2)T} \delta^{(2)}) \odot \varphi'(z^{(1)}) \partial L / \partial W^{(1)} = \delta^{(1)} x^T\end{aligned}$$

Update: $W \leftarrow W - \eta \cdot \partial L / \partial W$. Riutilizzando i calcoli intermedi, il costo di un passo è $\sim 2 \times$ il forward, indipendentemente dalla profondità: è ciò che rende addestrabili gli MLP.

Q9. Cos'è una funzione di costo e quale ruolo svolge nel processo di apprendimento? Fai un esempio con la MSE.

Una **funzione di costo** $L(\theta)$ misura quanto le predizioni del modello si discostano dai target sul training set. L'apprendimento si formula come $\theta^* = \operatorname{argmin}_{\theta} L(\theta)$ e la si minimizza con la discesa del gradiente ($\nabla_{\theta} L$ indica come aggiornare i parametri). **Esempio - MSE** (regressione):

$$L(\theta) = (1/n) \sum_i (f_{\theta}(x_i) - t_i)^2$$

Proprietà: differenziabile (gradiente $2(\hat{y} - t)$, ideale per backprop); penalizza quadraticamente gli errori grandi (sensibile agli outlier); minimizzarla equivale alla massima verosimiglianza con rumore gaussiano. Per la classificazione si usa invece la cross-entropy (Q29).

Q10. Qual è il significato del teorema di approssimazione universale per le reti neurali?

Una rete con **un solo layer nascosto** e attivazione non polinomiale, con abbastanza neuroni, può approssimare **qualunque funzione continua** su un compatto con precisione arbitraria:

$$F(x) = \sum_i \alpha_i \varphi(w_i \cdot x + b_i), \sup_K |F(x) - f(x)| < \varepsilon$$

Significato: gli MLP sono **approssimatori universali**, nessuna funzione continua è fuori portata (a differenza dell'SLP, Q6). **Cosa NON dice:** quanti neuroni servono (può essere enorme), come trovare i pesi (è un risultato di sola esistenza), nulla sulla generalizzazione. In pratica le reti **profonde** ottengono lo stesso con molti meno neuroni.

Q11. Quali sono i componenti fondamentali di un "perceptrone" o "neurone artificiale", e a cosa servono i "pesi" e il "bias"?

Componenti: **ingressi** x , **pesi** w , **bias** b , **attivazione** φ . Calcolo:

$$z = w \cdot x + b, y = \varphi(z)$$

- **Pesi:** misurano l'importanza di ogni input (w_i grande $\rightarrow$ input influente; $w_i < 0 \rightarrow$ relazione inversa). w è il vettore normale all'iperpiano di decisione $w \cdot x + b = 0$.
- **Bias:** **trasla** l'iperpiano; senza, passerebbe sempre per l'origine, limitando le funzioni rappresentabili. Rende la decisione **affine** invece che lineare.

L'apprendimento consiste nel trovare w e b corretti.

Q12. Definisci la "funzione di costo" e illustra la differenza tra "loss L1" (geometria del taxi) e "loss L2" (MSE).

Funzione di costo: $L(\theta)$ che misura l'errore medio del modello (vedi Q9).

- **L2 (MSE):** $\ell = (\hat{y} - t)^2$, per vettori $\|\hat{y} - t\|_2^2$. Distanza **euclidea**.
- **L1 (MAE):** $\ell = |\hat{y} - t|$, per vettori $\|\hat{y} - t\|_1$. Distanza **Manhattan** (taxi).

	L1 (MAE)	L2 (MSE)
Crescita errore	lineare	quadratica
Differenziabile in 0	no	sì
Gradiente	± 1 costante	$2(\hat{y} - t)$
Outlier	robusta	sensibile
Stimatore ottimo	mediana	media

L2 amplifica gli errori grandi (buon segnale di convergenza ma sensibile agli outlier); L1 è robusta ma con gradiente costante. Compromesso: **Huber loss**.

Q13. Elenca e descrivi brevemente almeno quattro "funzioni di attivazione" comunemente utilizzate nelle reti neurali artificiali.

L'attivazione φ introduce non linearità (senza, la rete collasserebbe in una trasformazione lineare).

1. **Sigmoide** $\sigma(z) = 1/(1 + e^{-z}) \in (0,1)$: interpretabile come probabilità; satura (vanishing gradient).
Uso: output binario.
2. **tanh** $\in (-1,1)$: come la sigmoide ma centrata in 0 $\rightarrow$ gradienti più bilanciati. Uso: hidden layer RNN.
3. **ReLU** $\max(0, z)$: nessuna saturazione per $z > 0$, calcolo banale, sparsità. Standard negli hidden layer.
4. **Softmax** $e^{z_i} / \sum_j e^{z_j}$: trasforma un vettore in distribuzione di probabilità (somma 1). Uso: output classificazione multiclasse.

(Varianti: Leaky ReLU, GELU.)

Q14. Qual è il principio su cui si basa l'algoritmo di "backpropagation" e come mai è importante per l'addestramento di MLP?

Si basa su due idee:

1. **Regola della catena:** una rete è una composizione di funzioni, quindi il gradiente è il prodotto delle derivate locali lungo gli strati.
2. **Calcolo all'indietro:** si propaga il "segnale di errore" dall'output all'input riusingo i calcoli intermedi:

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \varphi'(z^{(l)})$$

da cui $\partial L / \partial W^{(l)}$ e $\partial L / \partial b^{(l)}$. **Importanza:** calcola **tutti** i gradienti in tempo $O(P)$, come un solo forward pass (senza, servirebbero P valutazioni con differenze finite $\rightarrow$ training impossibile per reti con milioni di parametri). Funziona per qualunque architettura differenziabile e ha reso possibile il deep learning.

Q15. Cosa afferma il "Teorema di approssimazione universale"?

Afferma che una rete feedforward con **un solo layer nascosto** e attivazione non polinomiale (sigmoide, tanh, ReLU...), con un numero sufficiente di neuroni, può approssimare **qualsunque funzione continua su un compatto** con precisione arbitraria:

$$F(x) = \sum_i \alpha_i \varphi(w_i \cdot x + b_i), \sup_K |F(x) - f(x)| < \varepsilon$$

È la **giustificazione teorica** delle reti neurali come modelli generali. Garantisce l'esistenza di pesi buoni, ma **non** quanti neuroni servano, né che la discesa del gradiente li trovi, né la generalizzazione su dati nuovi. (Stesso contenuto di Q10.)

Q16. Descrivere le fasi principali del processo di "digitalizzazione delle immagini", specificando i concetti di "campionamento" e "quantizzazione".

La digitalizzazione converte un'immagine continua in una matrice di pixel in due fasi principali:

- **Campionamento:** discretizzazione del **dominio spaziale**. Si valuta l'immagine su una griglia $M \times N$ di punti (i pixel) $\rightarrow$ definisce la **risoluzione**. Campionare troppo poco causa **aliasing**; il teorema di Nyquist richiede frequenza di campionamento $\geq$ doppio della massima frequenza del segnale.
- **Quantizzazione:** discretizzazione del **codominio** (intensità). I valori continui vengono mappati su L livelli (tipicamente $L = 256$, 8 bit). Pochi livelli $\rightarrow$ **banding/posterizzazione**.

Risultato: immagine digitale $I \in 0, \dots, L-1^{M \times N \times C}$ ($C = 1$ grigi, $C = 3$ RGB).

Q17. Cosa si intende per immagine digitale? Spiega i concetti di campionamento, quantizzazione e spazio colorimetrico.

Un'immagine digitale è una matrice (tensore) di pixel $H \times W \times C$, ottenuta per digitalizzazione.

- **Campionamento:** discretizzazione spaziale su griglia $\rightarrow$ risoluzione (vedi Q16).
- **Quantizzazione:** discretizzazione dei valori di intensità su L livelli (vedi Q16).
- **Spazio colorimetrico:** il sistema con cui i numeri del pixel diventano colore:
- **RGB:** additivo, 3 canali, standard per i display;
- **Scala di grigi:** 1 canale, $Y = 0.299R + 0.587G + 0.114B$ (luminanza);
- **HSV:** tinta/saturazione/valore, utile per selezioni per colore;
- **YCbCr:** separa luminanza e cromaticità, sfruttato nella compressione (JPEG).

Q18. Come si utilizza un istogramma per operazioni di elaborazione su immagini in scala di grigi?

L'**istogramma** $h(k)$ conta i pixel con intensità k ; normalizzato $p(k) = h(k)/(MN)$ è la distribuzione delle intensità. Rivela esposizione e contrasto (stretto = basso contrasto; spostato = sotto/sovra-esposto). Operazioni:

- **Stretching del contrasto:** rimappa il range usato $[a, b]$ su $[0, L - 1]$.
- **Equalizzazione:** usa la CDF come trasformazione, $I' = (L-1) \cdot CDF(I)$ → istogramma ~uniforme, massimizza il contrasto.
- **Sogliaatura (thresholding):** scelta una soglia T (anche automatica, Otsu) si binarizza per separare oggetto/sfondo.

Limite: l'istogramma **ignora la struttura spaziale** (due immagini diverse possono avere lo stesso istogramma).

Q19. Spiega come funziona il filtro di Sobel e quale informazione estrae da un'immagine.

Il **Sobel** approssima il **gradiente** dell'intensità per estrarre i **bordi**. Usa due kernel 3×3 :

$$S_x = \begin{vmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{vmatrix} \quad S_y = \begin{vmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{vmatrix}$$

Per convoluzione si ottengono $G_x = I * S_x$ e $G_y = I * S_y$, da cui:

$$M = \sqrt{G_x^2 + G_y^2} (\text{magnitudo : forza del bordo})$$

$$\theta = \arctan2(G_y, G_x) (\text{direzione del bordo})$$

M alta = bordo, bassa = zona uniforme. I pesi **(1,2,1)** fanno anche uno **smoothing** implicito → meno sensibile al rumore del Prewitt. È il secondo stadio della pipeline di Canny (progetto d'esame).

Q20. Qual è la differenza tra convoluzione e cross-correlazione? Fornisci una definizione formale.

Cross-correlazione (2D): il kernel scorre e si applica direttamente:

$$(I \star K)[i, j] = \sum_{u, v} I[i + u, j + v] \cdot K[u, v]$$

Convoluzione (2D): come sopra ma con il **kernel ribaltato** (segno meno):

$$(I * K)[i, j] = \sum_{u, v} I[i - u, j - v] \cdot K[u, v]$$

Differenza: la convoluzione ribalta il kernel di 180° prima di applicarlo; per kernel simmetrici coincidono. Solo la **convoluzione** è commutativa e associativa (e soddisfa il teorema di Fourier). Le CNN, nonostante il nome, usano la cross-correlazione: i pesi sono appresi, quindi il flip è irrilevante.

Q21. Quali sono le proprietà principali dell'operazione di convoluzione?

1. **Commutativa:** $f * g = g * f$
2. **Associativa:** $(f * g) * h = f * (g * h) \rightarrow$ permette di comporre filtri
3. **Distributiva** sulla somma: $f * (g + h) = f * g + f * h$
4. **Lineare:** $(\alpha f) * g = \alpha(f * g)$
5. **Elemento neutro:** la delta di Dirac, $f * \delta = f$
6. **Differenziazione:** $(f * g)' = f' * g = f * g'$ (es. derivata della gaussiana)
7. **Teorema di convoluzione:** $Ff * g = Ff \cdot Fg \rightarrow$ in frequenza diventa prodotto, calcolo via FFT
8. **Invarianza per traslazione** $\rightarrow$ base dell'equivarianza delle CNN
9. **Separabilità:** se $K = Kx \cdot Ky$, costo da $O(k^2)$ a $O(k)$ (es. gaussiana)

Q22. Illustra l'utilizzo dell'"istogramma dei livelli di grigio" in relazione agli "operatori puntuali" per la manipolazione di immagini luminanza.

Un **operatore puntuale** trasforma ogni pixel in funzione del solo suo valore: $I'[i, j] = T(I[i, j])$, senza guardare i vicini. L'effetto sull'istogramma è prevedibile: se T è iniettiva $h'(T(k)) = h(k)$. Principali operatori (definiti tramite T):

- **Negativo:** $T(k) = (L - 1) - k$
- **Correzione gamma:** $T(k) = (L - 1)(k/(L - 1))^\gamma$ ($\gamma < 1$ schiarisce le ombre)
- **Contrast stretching:** $T(k) = (L - 1)(k - a)/(b - a)$
- **Equalizzazione:** $T(k) = (L - 1) \cdot CDF(k) \rightarrow$ istogramma uniforme
- **Sogliatura:** $T(k) = 0$ se $k < T$, altrimenti $L - 1$

Si implementano via **lookup table** ($O(1)$ per pixel). Limite: ciechi al contesto spaziale (non riducono rumore, non rilevano bordi).

Q23. In che modo i "polinomi di Taylor" vengono applicati per la creazione di "filtri di sharpening" come il filtro di Sobel o il filtro Laplaciano?

Le derivate discrete di un'immagine si ricavano dallo **sviluppo di Taylor** dei pixel vicini. **Derivata prima** (Sobel): sottraendo gli sviluppi di $I[i, j + 1]$ e $I[i, j - 1]$:

$$\partial I / \partial x \approx (I[i, j + 1] - I[i, j - 1]) / 2 \rightarrow \text{kernel}[-1, 0, +1]$$

mediato sulle tre righe con pesi (1,2,1) dà il kernel di Sobel. **Derivata seconda** (Laplaciano): sommando gli sviluppi:

$$\partial^2 I / \partial x^2 \approx I[i, j + 1] - 2I[i, j] + I[i, j - 1]$$

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2 \rightarrow \text{kernel}[[0, 1, 0], [1, -4, 1], [0, 1, 0]]$$

Sharpening: $I_{sharp} = I - k \cdot \nabla^2 I$ (accentua i bordi). In sintesi: i filtri differenziali sono **polinomi di Taylor discretizzati** sui vicini di un pixel.

Q24. Definisci l'operazione di convoluzione nel continuo e indicane le principali proprietà, sia in una che in due variabili.

1 variabile:

$$(f * g)(t) = \int f(\tau)g(t - \tau)d\tau$$

2 variabili:

$$(f * g)(x, y) = \iint f(u, v)g(x - u, y - v)dudv$$

Concettualmente: si fa scorrere **g** (riflessa) su **f** e si integra il prodotto (media pesata). Proprietà: commutativa, associativa, distributiva, lineare; elemento neutro δ ; differenziazione $(f * g)' = f' * g$; teorema di convoluzione (Fourier $\rightarrow$ prodotto); invarianza per traslazione; separabilità in 2D. (Vedi Q21.) È un'operazione **lineare e invariante per traslazione**, base di filtri, sistemi LTI e CNN.

Q25. Quali sono le differenze strutturali e funzionali tra un MLP e una CNN?

Aspetto	MLP	CNN
Connettività	fully-connected	locale (kernel)
Pesi	indipendenti	condivisi
N. parametri	altissimo	ridotto
Input	vettore 1D	tensore $H \times W \times C$
Equivarianza traslazione	no	sì

L'MLP appiattisce l'immagine perdendo la struttura spaziale; la CNN preserva la griglia 2D ed estrae feature locali con pesi condivisi. Su un'immagine $224 \times 224 \times 3$, un MLP avrebbe $\sim 10^8$ parametri nel primo strato, una CNN poche migliaia. Una CNN è di fatto un MLP con vincoli (località + condivisione pesi) che incorporano l'**inductive bias** delle immagini: molti meno parametri ed equivarianza per traslazione.

Q26. Perché le "Convolutional Neural Networks (CNN)" sono particolarmente efficaci per la gestione di "dati a griglia", come le immagini?

Sfruttano per costruzione tre proprietà dei dati a griglia:

1. **Località:** i pattern (bordi, texture) sono in regioni locali $\rightarrow$ kernel piccoli; strati profondi vedono regioni più grandi (campo recettivo crescente).
2. **Condivisione dei pesi:** lo stesso kernel ovunque $\rightarrow$ **equivarianza per traslazione**, pochi parametri (un kernel 3×3 ha $9 \cdot C \cdot C'$ pesi indipendentemente dalla risoluzione), generalizzazione spaziale.
3. **Composizionalità gerarchica:** con conv + pooling si costruiscono feature sempre più astratte (bordi $\rightarrow$ parti $\rightarrow$ oggetti).

Imporre queste simmetrie come inductive bias (invece di farle apprendere a un MLP) riduce drasticamente i parametri e migliora la generalizzazione.

Q27. Dare una breve descrizione dei minimi esempi di strati utilizzati in una CNN: "layer convoluzionale", "funzione di attivazione ReLU", "layer di pooling", e "dropout".

- **Convoluzionale:** applica K filtri appresi (es. 3×3) all'input $\rightarrow$ feature map. $y[i, j, k] = \sum W_k[u, v, c] \cdot x[i + u, j + v, c] + b_k$. Estrae feature locali. Iperparametri: kernel size, stride, padding, n. filtri.
- **ReLU:** $\max(0, z)$ element-wise dopo la conv. Introduce non linearità, niente vanishing gradient per $z > 0$, induce sparsità.
- **Pooling** (es. $\max 2 \times 2$): riduce la risoluzione spaziale $\rightarrow$ meno calcolo, invarianza a piccole traslazioni, campo recettivo maggiore. Senza parametri.
- **Dropout:** in training spegne ogni neurone con probabilità p (e riscalda di $1/(1 - p)$). **Regolarizzazione** anti-overfitting (effetto ensemble). Disattivato in inference.

Stack tipico: $Conv \rightarrow ReLU \rightarrow Pool \rightarrow \dots \rightarrow FC \rightarrow Dropout \rightarrow FC \rightarrow Softmax$.

Q28. Per quale tipo di dati sono concepite le "Recurrent Neural Networks (RNN)" e quale problematica affrontano?

Le **RNN** sono concepite per **dati sequenziali** (testo, audio, serie temporali, video) in cui l'ordine conta e ogni elemento dipende dal contesto precedente. Risolvono due limiti delle reti feedforward sulle sequenze: input di lunghezza variabile e assenza di memoria. Introducono uno **stato nascosto** aggiornato a ogni passo:

$$h_t = \varphi(W_h h_{t-1} + W_x x_t + b)$$

h_t riassume il passato; i pesi sono **condivisi nel tempo**. Si addestrano con la backpropagation through time. **Problema principale: vanishing/exploding gradient** su lunghe sequenze (il gradiente attraversa T moltiplicazioni per W_h) $\rightarrow$ non imparano dipendenze a lungo termine. Soluzione: **LSTM/GRU**, che con dei gate mantengono selettivamente la memoria.

Q29. Quali sono le principali caratteristiche della "funzione di costo cross-entropy" e della "funzione di attivazione softmax", e in quale contesto sono spesso utilizzate?

Coppia standard per la **classificazione multiclasse**. **Softmax** trasforma i logit in distribuzione di probabilità:

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}, \sum_i = 1$$

Generalizza la sigmoide a K classi; invariante per costante additiva (si sottrae $\max(z)$ per stabilità). **Cross-entropy** misura la distanza dalla distribuzione vera; con target one-hot:

$$L = - \sum_i t_i \log p_i = - \log p_c (c = \text{classe vera})$$

È la negative log-likelihood (massima verosimiglianza). Insieme danno un **gradiente pulito**: $\partial L / \partial z_i = p_i - t_i$. Uso: classificazione multiclasse, modelli di linguaggio (next token). Per $K = 2 \rightarrow$ binary cross-entropy con sigmoide.

Q30. Descrivi l'approccio dei modelli "Generative Adversarial Networks (GAN)".

Le **GAN** sono modelli generativi basati su due reti in **competizione**:

- **Generatore G**: da rumore $\mathbf{z}$ genera dati falsi $G(\mathbf{z})$, vuole ingannare D.
- **Discriminatore D**: stima la probabilità che un campione sia reale.

Si addestrano in un **gioco minimax**:

$$\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(\mathbf{z})))]$$

D massimizza (distinguere vero/falso), G minimizza (produrre campioni credibili). All'equilibrio $p_G = p_{\text{data}}$ e $D = 1/2$ (non distingue più). **Difficoltà**: addestramento instabile, **mode collapse** (G genera pochi campioni ripetitivi), vanishing gradient per G se D è troppo bravo (si usa la non-saturating loss $-\log D(G(\mathbf{z}))$).

Q31. Quali problemi possono sorgere nella retropropagazione in MLP profondi (vanishing/exploding gradients)?

Nella backprop il gradiente di uno strato profondo è il **prodotto** di molte Jacobiane $J^{(k)} = W^{(k)T} \text{diag}(\varphi'(z^{(k)}))$. Se la norma di questa catena cresce o decresce esponenzialmente con la profondità:

- **Vanishing:** $\|\partial L/\partial W\| \rightarrow 0$, gli strati iniziali non imparano. Cause: attivazioni saturanti (sigmoide/tanh, $\varphi' \approx 0$), pesi piccoli.
- **Exploding:** $\|\partial L/\partial W\| \rightarrow \infty$, aggiornamenti enormi, divergenza/NaN. Cause: pesi grandi, RNN su lunghe sequenze.

Soluzioni: inizializzazione **Xavier/He**; attivazioni non saturanti (**ReLU**); **Batch Normalization**; **skip/residual connections** (ResNet); **gradient clipping** (per l'exploding); architetture con gate (LSTM); optimizer adattivi (Adam). Il deep learning moderno è in gran parte l'insieme di queste tecniche.

Q32. Spiega il funzionamento di una CNN, indicando il ruolo di ciascun tipo di layer (convoluzionale, pooling, lineare).

Una **CNN** trasforma un'immagine $H \times W \times C$ in un output (es. classi) tramite una **gerarchia di feature**: bordi $\rightarrow$ parti $\rightarrow$ oggetti. Ruolo dei layer:

- **Convoluzionale**: estrae **feature locali** con filtri condivisi ($y = \sum W_k \cdot x + b_k$). Località + condivisione pesi $\rightarrow$ equivarianza e pochi parametri. Seguito da ReLU.
- **Pooling**: **down-sampling** spaziale; riduce calcolo, dà invarianza a piccole traslazioni, aumenta il campo recettivo. Senza parametri.
- **Lineare (fully-connected)**: alla fine, combina globalmente le feature per la **classificazione** (+ softmax). Sta in fondo perché ha molti parametri e lavora su feature già compatte.

Struttura tipica:

Input $\rightarrow$ [Conv $\rightarrow$ ReLU $\rightarrow$ Pool] $\times$ N $\rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Softmax

Fine delle 32 domande.